Semester B, 2025-2026

City University of Hong Kong

CS4486

Artificial Intelligence

Project

Topic: YouTube Spam Classifier

Name: Shek King Fung

Student ID: 58527549

## 1. *Background*

YouTube is one of the largest video-sharing platforms in the world, with more than billions of users and generating millions of comments daily. The comment section is an important part of it for users to express their thoughts and discuss with others. However, apart from these meaningful comments, YouTube also suffers from spam comments. Those spam comments are full of irrelevant, promotional, or malicious content. This has not only cluttered the comments section but also undermines user experience, and might even pose security risk [1], [2].

YouTube has its own spam filtering system, but many spam comments still manage to bypass it [1]. For solving this problem, a machine learning based classifier for detecting and filtering spam comments might be needed.

The goal of this project is to build a machine learning classifier that can automatically distinguish between spam comments (label as 1) and non-spam comments (label as 0). The project compares four different classification algorithms, evaluating their performance on a dataset provided, and selecting the best performance one.

The data set used in this project was provided by the course instructor. It consists of a training set, with 798 labeled comments in CSV format, and a test set, with 202 labeled comments in Excel format. Both files contain comment text ("TEXT" column) and class labels ("CLASS" column).

## 2. *Method*

### 2.1. *Text Cleaning*

Raw text cannot be used directly by machine learning models. The text first needs to be cleaned before passing to the model for processing. In this report, pandas is used for text cleaning, and it first converts all letters to lowercase. Then remove the byte order mark, URL, punctuation mark, special symbols, numbers and extra spaces. These steps help the model focus on meaningful words instead of noise.

### 2.2. *TF-IDF Feature Extraction*

After cleaning, we need to extract features from text. TF-IDF (Term Frequency-Inverse Document Frequency) is used in this project and is done by calling the

TfidfVectorizer from scikit. This method gives higher weight to words that appear often in one comment, but rarely in other comments. This helps capture important keywords, which is important for spam detection.

The following settings are used in TfidfVectorizer:

- max_features = 3000: only keep the 3000 most common words
- ngram_range = (1,2): look at single words and pairs of words
- stop_words = 'english': remove common words like "the", "a", "is"

After this step, the training data becomes a matrix with 798 rows and 3000 columns.

### *2.3. Four Chosen Models*

Four models from [1], [2] are chosen, they are commonly used for text classification. They are Logistic Regression, Multinomial Naive Bayes, Random Forest, and Multi-Layer Perceptron. All models were implemented using Scikit-learn.

Logistic Regression is a simple linear model. It is fast and easy to implement and is often used as the baseline in classification.

Multinomial Naive Bayes is a probabilistic model that uses the Bayes' theorem, which assumes each word is independent. It works well for text data and is very fast.

Random Forest is an ensemble method. It builds many decision trees and combines their results. It can capture complex patterns and usually perform best for classification.

Multi-Layer Perceptron (MLP) is a simple neural network. The neural network for this model has one hidden layer with 100 neurons. It can learn non-linear relationships but takes long time to train.

### *2.4. Evaluation*

Several metrics and methods are used to evaluate model performance. Including:

- Cross-validation: This is done by calling the cross_val_score in scikit and the split number is 5 in this report. It splits the training data into 5 parts. Each time, 4 parts are used for training and 1 part for validation. This gives a more stable estimate of performance.

- Training accuracy: How well the model performs on the training data. If this is much higher than test accuracy, overfitting happens.

- Test accuracy: How well the model performs on test data. This is the most important metric.

- Weighted F1-score: Calculates the average F1-score, which combines precision and recall, of each class weighted by the number of samples in that class. It is useful when classes are not perfectly balanced.

- Confusion matrix: This shows exactly how many comments were correctly or incorrectly classified (true positives, true negatives, false positives, and false negatives).

- Training time: How long it takes to train the model. It is important for real-world applications.

## *3. Experiments*

### *3.1. Model Comparison Results*

All four models are trained and the results are compared. The results are shown below. The best performance model will be further improved by hyperparameter tuning.

```
---------------Logistic Regression----------------
  Cross-validation accuracy: 0.7920
  Training accuracy: 0.9536
  Test accuracy: 0.8465
  Weighted F1-score: 0.8423
  Training time: 0.01 seconds
  Confusion Matrix (test set):
[[ 64  28]
 [  3 107]]
  Classification Report (test set):
              precision    recall  f1-score   support

         Ham       0.96      0.70      0.81        92
        Spam       0.79      0.97      0.87       110

    accuracy                           0.85       202
   macro avg       0.87      0.83      0.84       202
weighted avg       0.87      0.85      0.84       202


--------------------------------------------------
```

```
-------------------Naive Bayes--------------------
  Cross-validation accuracy: 0.8284
  Training accuracy: 0.9549
  Test accuracy: 0.8267
  Weighted F1-score: 0.8206
  Training time: 0.00 seconds
  Confusion Matrix (test set):
[[ 60  32]
 [  3 107]]
  Classification Report (test set):
              precision    recall  f1-score   support

         Ham       0.95      0.65      0.77        92
        Spam       0.77      0.97      0.86       110

    accuracy                           0.83       202
   macro avg       0.86      0.81      0.82       202
weighted avg       0.85      0.83      0.82       202


--------------------------------------------------



------------------Random Forest-------------------
  Cross-validation accuracy: 0.8108
  Training accuracy: 0.9762
  Test accuracy: 0.8564
  Weighted F1-score: 0.8530
  Training time: 0.44 seconds
  Confusion Matrix (test set):
[[ 66  26]
 [  3 107]]
  Classification Report (test set):
              precision    recall  f1-score   support

         Ham       0.96      0.72      0.82        92
        Spam       0.80      0.97      0.88       110

    accuracy                           0.86       202
   macro avg       0.88      0.85      0.85       202
weighted avg       0.87      0.86      0.85       202


--------------------------------------------------



-----------------------MLP------------------------
  Cross-validation accuracy: 0.8309
  Training accuracy: 0.9762
  Test accuracy: 0.8416
  Weighted F1-score: 0.8390
  Training time: 7.22 seconds
  Confusion Matrix (test set):
[[ 67  25]
 [  7 103]]
  Classification Report (test set):
              precision    recall  f1-score   support

         Ham       0.91      0.73      0.81        92
        Spam       0.80      0.94      0.87       110

    accuracy                           0.84       202
   macro avg       0.86      0.83      0.84       202
weighted avg       0.85      0.84      0.84       202


--------------------------------------------------
```

Finding from the experiment:

Random Forest gave the highest test accuracy among the four models, reaching 85.64%. However, there is a big gap between training accuracy (97.62%) and test accuracy (85.64%). This means overfitting happens. It memorized the training data too well and can't recognize new patterns.

Logistic Regression was extremely fast. It trained in only 0.01 seconds and still achieved 84.65% test accuracy, which is the second highest. If both accuracy and speed need to be considered, this model is a good choice.

MLP took much longer to train (7.22 seconds) but did not give better results. For this dataset, a simple neural network does not perform well and takes long time to train.

Naive Bayes was the fastest but had the lowest test accuracy (82.67%).

It is also observed that the test accuracy difference between four models is less than 5%. This means that when the dataset is relatively small, the performance between models is small.

### *3.2. Improving Random Forest with Hyperparameter Tuning*

Random Forest performed best (highest test accuracy) among the four models, so it was chosen for hyperparameter tuning to see if it could be further improved. RandomizedSearchCV from scikit is used to test different parameters combinations. The parameters test include:

- n_estimators: [50, 100, 150, 200, 300] (How many trees)
- max_depth: [5, 10, 15, 20, 25, 30, None] (How deep each tree can grow)
- min_samples_split: [2, 5, 10, 15, 20] (Minimum samples required to split a node)
- min_samples_leaf: [1, 2, 3, 4, 5] (Minimum samples at a leaf node)
- max_features: ['sqrt', 'log2', None] (How many features to consider at each split)

The best parameters found were:

- n_estimators = 50
- max_depth = 15
- min_samples_split = 20
- min_samples_leaf = 4
- max_features = None

After tuning, the optimized Random Forest performed are as follows:

```
============================================================
Hyperparameter Tuning for Random Forest
============================================================
Best parameters: {'n_estimators': 50, 'min_samples_split': 20, 'min_samples_leaf': 4, 'max_features': None, 'max_depth': 15}
Best cross-validation accuracy: 0.8094

Optimized Random Forest train accuracy: 0.8409
Optimized Random Forest test accuracy: 0.8663
Optimized Random Forest test weighted F1-score: 0.8664


Confusion Matrix (optimized Random Forest):
[[87  5]
 [22 88]]

Classification Report (optimized Random Forest):
              precision    recall  f1-score   support

         Ham       0.80      0.95      0.87        92
        Spam       0.95      0.80      0.87       110

    accuracy                           0.87       202
   macro avg       0.87      0.87      0.87       202
weighted avg       0.88      0.87      0.87       202
```

And the difference between before and after tuning for Random Forest:

```
============================================================
Default vs Optimized Random Forest Comparison
============================================================
Default RF test accuracy: 0.8564
Optimized RF test accuracy: 0.8663
Improvement: 0.0099 (+0.99%)
```

It is also needed to note that the training accuracy has dropped from 97.62% to 84.09%. This means that even though the tuning Random Forest performs better in test data, it performs worse in training data. But that could also mean that the overfitting problem for Random Forest is handled using the tuning parameter.

### 3.3. *Top 10 Most Important Words for Spam Detection*

One advantage of Random Forest is that it can tell which features (words) are the most important for making predictions. This means with these words appear, it is likely to be a spam comment.

```
============================================================
Top 10 Most Important Words for Spam Detection
============================================================
1. 'check' (importance: 0.2031)
2. 'subscribe' (importance: 0.1854)
3. 'channel' (importance: 0.0799)
4. 'views' (importance: 0.0631)
5. 'money' (importance: 0.0610)
6. 'help' (importance: 0.0514)
7. 'song' (importance: 0.0398)
8. 'free' (importance: 0.0373)
9. 'video' (importance: 0.0372)
10. 'love' (importance: 0.0249)
```

It can show that spam messages usually ask people to subscribe, check some channels or related to money.

### 3.4. *Confusion Matrix Analysis*

Since the optimized Random Forest has the highest test accuracy, so it will be focused on. The confusion matrix for optimized Random Forest is as follows:

```
Confusion Matrix (optimized Random Forest):
[[87  5]
 [22 88]]
```

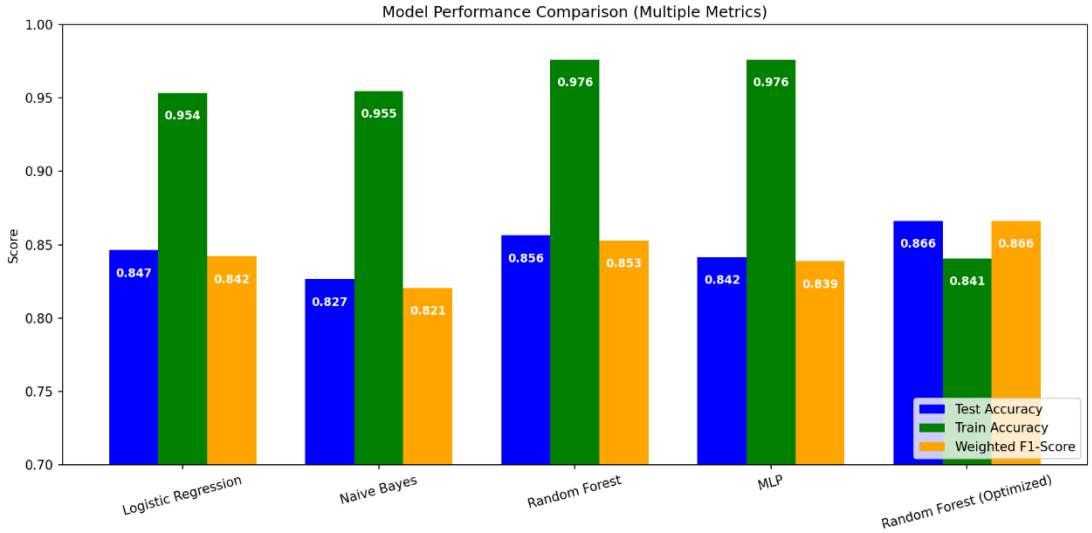
Consider spam as the positive class. There are 87 true negatives (actual normal, classified normal), 5 false positives (actual normal, wrongly classified as spam), 88 true positives (actual spam, classified spam), and 22 false negatives (actual spam, wrongly classified as normal).

This means the model caught 87 out of 92 normal comments (94.6% recall for normal) and 88 out of 110 spam comments (80.0% recall for spam), having an overall accuracy of 86.63%.

Compared to the default Random Forest, which caught 66 out of 92 normal comments (71.7% recall for normal) and 107 out of 110 spam comments (97.3% recall for spam). The optimized Random Forest got much better at recognizing normal comments, but the trade-off is that spam recall dropped slightly.

### *3.5. Visualization*

A bar chart is created to compare all models. The chart shows test accuracy, training accuracy, and weighted F1-score for each model. The optimized Random Forest performs best on both test accuracy and F1-score.




## *4. Conclusion*

This project compared four machine learning models for YouTube spam comment detection. It is discovered that the optimized Random Forest achieved the highest test accuracy at 86.63%. This is the best model among the models that are being tested. But the gap between models is slightly in these small data sets being trained and tested.

It is also found in the Random Forest that the most important words for detecting spam are promotional terms like "subscribe", "check", "channel", and "views". This matches what we see in real YouTube spam comments.

Logistic Regression is a good alternative if speed matters. It trains in less than 0.01 seconds and still gets 84.65% accuracy. For real-time applications, this might be the better choice.

## References


[1] H. Oh, "A YouTube Spam Comments Detection Scheme Using Cascaded Ensemble Machine Learning Model," *IEEE Access*, vol. 9, pp. 144121-144128, 2021, doi: 10.1109/ACCESS.2021.3121508.

[2] A. S. Xiao and Q. Liang, "Spam detection for Youtube video comments using machine learning approaches," *Mach. Learn. Appl.*, vol. 16, p. 100550, Jun. 2024, doi: 10.1016/j.mlwa.2024.100550.

[3] A. Mahesh, "Detecting spam comments on YouTube using Machine Learning," *Medium*, Sep. 2020. [Online]. Available: https://medium.com/@akshmahesh/detecting-spam-comments-on-youtube-using-machine-learning-948d54f47b3. [Accessed: 22 Apr. 2026].

[4] K. Flitcher, "YouTube comments Spam Classifier," *Medium*, May 2019. [Online]. Available: https://medium.com/@koushik.flitcher/youtube-comments-spam-classifier-632299fb051f. [Accessed: 22 Apr. 2026].

[5] Scikit-learn, "Scikit-learn: Supervised learning," *scikit-learn.org*. [Online]. Available: https://scikit-learn.org/stable/supervised_learning.html. [Accessed: Apr. 24, 2026].